03 eSignet | Enable Native SSO in eSignet Go for Mobile Apps

Implement OpenID Connect Native SSO in eSignet (Go) to enable seamless, secure single sign-on across multiple native apps signed by the same vendor, installed on the same device.

Complexity Level: **Medium**

Background

What is Single Sign-On (SSO) for Mobile Apps?

It is an extension to the standard OpenID Connect Authentication flow that enables a second mobile app to share the authentication of the first mobile app, where both mobile applications are signed by the same vendor certificates. This is the essence of Single Sign-On (SSO): one trusted login, reused securely across all mobile apps in the same family. For this to work safely on mobile, the “session” cannot simply be a shared browser cookie (mobile apps don’t share those); it must be a deliberately designed, cryptographically secured handoff of authentication state between apps on the same device, which is what Native SSO defines.

eSignet has recently migrated to Golang and, through a collaboration with the WSO2 team, now leverages select components from the Thunder ID Engine as part of its backend core. This migration brings two key benefits: improved runtime performance and easier long-term maintainability of the codebase. This new architecture already supports standard OIDC flows and the FAPI security profile. However, it currently lacks support for Native SSO across mobile apps, a capability that is a valid and increasingly common requirement across MOSIP-based deployments.

Why We Want to Solve It

Mobile applications do not share browser cookies or session state the way web apps do, so within a mobile app ecosystem a user is typically forced to log in separately to every mobile app even when all apps belong to the same trusted vendor family and the user has already proven their identity once. This repeated authentication creates unnecessary friction and a poor user experience.

Solving this improves user experience by removing repeated logins, reduces authentication friction across a mobile app suite, and creates a more standards-based, secure way to reuse a verified user session across apps. It is especially useful in eSignet/MOSIP ecosystems where multiple mobile apps may need to rely on the same trusted identity.

User Journey

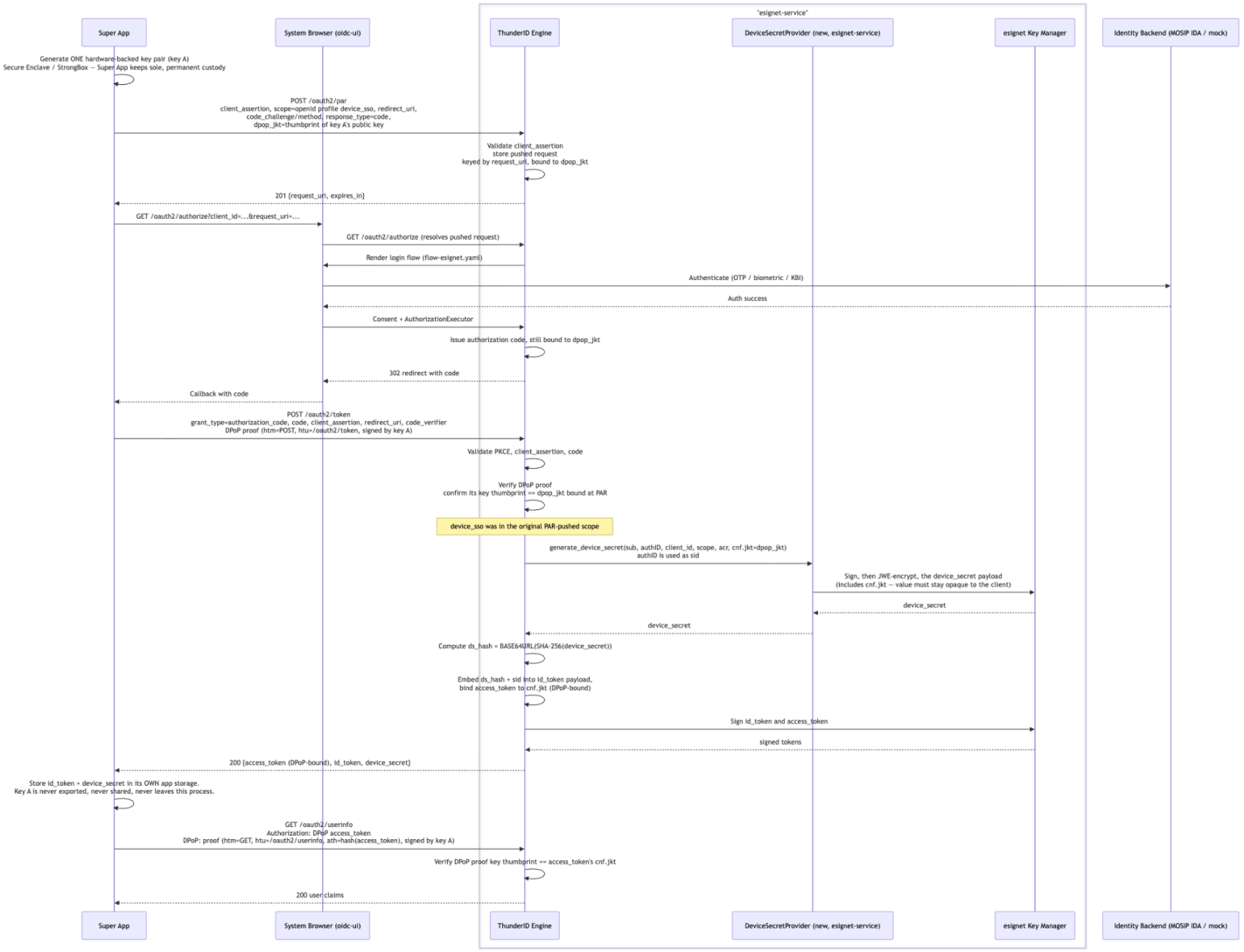
For logging into the first mobile App, the user will experience the following journey:

1. User opens the first App on their Android/iOS device.
2. On the login page, the user sees a **“Sign in with eSignet”** button.
3. User taps **“Sign in with eSignet”** and is redirected to the eSignet authentication UI on mobile browser.
4. User selects their preferred authentication factor to proceed.
    a. For this flow, we assume the user chooses **“Login with OTP.”**
5. User is prompted to enter their UIN and tap **“Send OTP.”**
6. User receives the OTP, enters it on the next screen, and taps **“Submit.”**
7. On first login, the user is shown a consent screen listing the claims (user PII data) and authorization scopes requested by the Relying Party (the first App).
8. User reviews the requested claims, makes their selection, and taps **“Allow.”**
9. User is logged in and lands on the first mobile App home screen (depends on the redirect_uri).
10. User opens second mobile app(by the same vendor as first mobile app) and taps **“Sign In.”**
11. User is **not** prompted to authenticate again, they are seamlessly signed in and land directly on the second mobile app’s home page.

This is the Native SSO experience: a single explicit authentication (Steps 3–8) on the one mobile App is silently and securely reused to sign the user into any other mobile app within the same vendor family (Steps 10–11), without repeating login or consent.

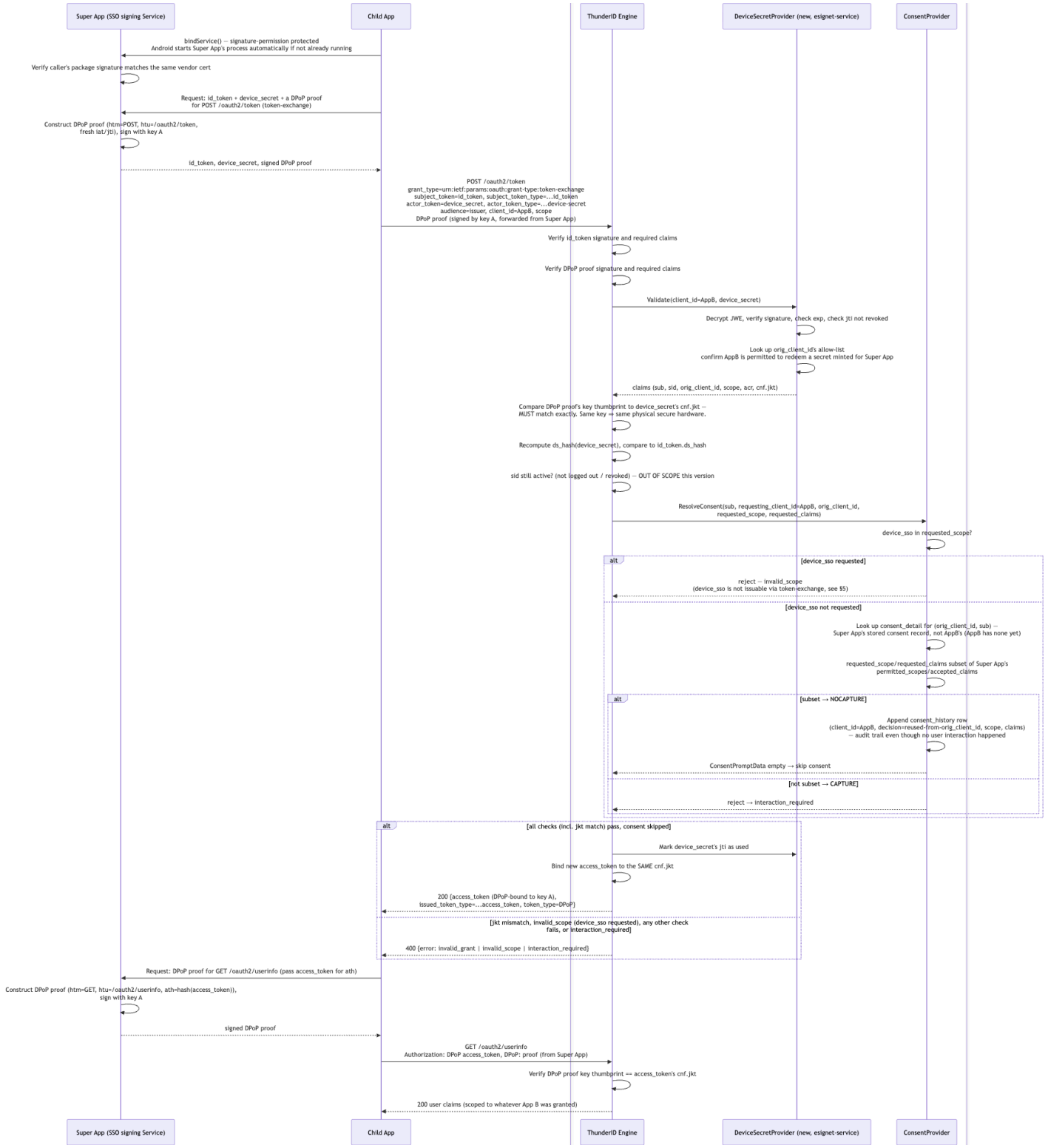
Sequence Diagram

1. Super APP Login Flow - PAR + DPoP Login and Device Secret Issuance



Super APP Login Flow

2. Child App Login Flow – Silent SSO via Token Exchange, Super App Signs For Child Apps



Child App Login Flow

Expected Tasks

Since the goal is to implement Native SSO for mobile apps on top of the existing OIDC login flow secured with FAPI security profile, below are the mandatory tasks required on top of it to achieve the SSO feature. Since eSignet is leveraging the Thunder ID Engine as part of its backend core, the required changes span both repositories. The respective changes are marked with a label against each task.

Prerequisite

1. Create two native Android/iOS apps signed with the same certificate.
2. Integrate the "Sign in with eSignet" button into the first mobile App, following the Relying Party integration guide.

Mandatory Tasks

1. Request the `device_sso` scope *[eSignet]*
    a. Add support for a new `device_sso` scope to distinguish a Native-SSO-enabled login from a normal authentication request; this scope is added to the client's authorize request.

2. Validate the scope at the token endpoint *[eSignet]*
    a. Implement the token endpoint check that validates whether the `device_sso` scope is enabled for the requesting registered client.
        i. Confirming `allowed_authorization_scopes` includes `device_sso`.

3. Implement the Device Secret Provider interface *[Thunder ID]*
    a. Implement the Device Secret Provider interface, to be exposed for use by the Thunder ID Engine.

4. Add below 2 methods to the Device Secret Provider interface *[eSignet]*
    a. **Generate Device Secret**: Generates a signed JWT as `device_secret`. This JWT must include:
        i. `cnf.jkt`
        ii. `auth_id`
        iii. Expiry time
        iv. `client_id`
        v. `scope`
        vi. `acr`
        vii. `claims`
        viii. `iss`
        ix. The `device_secret` must be embedded into the `id_token` returned by the token endpoint.

b. **Validate Device Secret**: Validates a device secret presented by the second App during token exchange.

5. **Invoke device secret generation from the token endpoint.**
   a. Once the `device_sso` scope is found valid for the client, invoke the **device secret generation** method of the **Device Secret Provider interface.**

6. **Return tokens and device secret to the Super App** ***[eSignet]***
   a. The token endpoint response to the first App must return the below details. These should be persisted for reuse in the second App sign-in flow:
      i. `access_token`
      ii. `id_token`
      iii. `device_secret` (signed-then-encrypted JWT)

7. **Retrieve the `id_token` and `device_secret` in the second App** ***[Client, Mobile App]***
   a. Add support for retrieving the `id_token` and `device_secret` in the second App from the first App via an app-to-app intent, once the second App is opened post first App login. Details to be fetched:
      i. `id_token`, given as response to the second App
      ii. `device_secret`, given as response to the second App
      iii. DPoP proof

8. **Initiate the token exchange call** ***[Client, Mobile App]***
   a. Add support to call the token endpoint with `grant_type=token_exchange`, having the second App reuse the first App's key to sign its DPoP proof.

      Note: The `device_sso` scope must not be included in the second App's token exchange endpoint call.

9. **Handle the Token Exchange grant at the token endpoint** ***[Thunder ID Engine]***
   a. Add support for the OAuth 2.0 Token Exchange grant type at the token endpoint, including:
      i. Accepting `actor_token_type=urn:openid:params:token-type:device-secret`.
      ii. Validating the `actor_token` by calling the Device Secret Provider's Validate Device Secret method.

iii. Validating the `id_token`, including:
    1. Confirming the binding between the `id_token` and the device secret via `ds_hash`.
    2. Validating the session ID (`sid`).
    3. Confirming allowed_authorization_scopes includes `device_sso`.
    4. Confirming the `cnf.jkt` in the device secret matches the key thumbprint in the current request's DPoP header.

10. **Enforce scope rules for the second App**
    a. Enforce that the scope requested by the second App is a subset of the scope originally granted to the first App, e.g., if the second App requests the `profile` scope but the first App only requested the `email` scope, the exchange must fail; the reverse (second App requesting a narrower scope than the first App) is allowed.
    b. Reject any token endpoint call with “token_exchange” grant that includes the `device_sso` scope, this scope is valid only on the token endpoint with “authorization_code” grant.

Bonus Tasks

1. Trigger a re-consent screen in the second App when the scopes requested during its token exchange differ from what was originally consented to in the first App.

Deliverables

1. Native mobile Apps, developed to demonstrate the complete flow in a UI.
2. Updated Postman collection reflecting the new Native SSO endpoints and flows.
3. Documentation including:
    - Updated API documentation.
    - Design document based on MOSIP/eSignet technical design and RFC behavior.
4. PPT presentation summarizing the approach and implementation.
5. Demo video showing the solution end to end.

Recommended Tech Stack

- **Backend:** Golang
- Mobile Apps:
    - **Android:** Kotlin/Java, C or C++
    - **iOS:** Objective-C or Objective-C++, Swift

Resources

Refer to the links below for a better understanding of RFCs mentioned in the document:

1. FAPI Security Profile
2. OpenID Connect Core
3. OpenID Connect Native SSO for Mobile Apps 1.0

Source Code

1. eSignet Go
2. Thunder ID

Documentation

1. eSignet Docs
2. eSignet Go Architecture
3. End User Guides

Sandbox to Experiment and Integrate

1. **MOSIP Collab Sandbox Environment:** The Collab Sandbox lets you explore and integrate with the MOSIP applications. It is the fastest path to a working end-to-end demo during the hackathon.
2. eSignet Discovery Endpoints

Community Forum

1. MOSIP Community

Register the Mobile Apps as OIDC Clients

1. Integrate with eSignet

Self-Register to Create a MOSIP ID

1. Self Registration Portal